```
// E-Commerce Frontend - React + Tailwind CSS
// Single-file implementation with all pages and components

import { useState, useEffect, useContext, createContext, useCallback } from "reac

// ─── DATA ────────────────────────────────────────────────────────────────────
const PRODUCTS = [
  { id: 1, name: "Marble Arc Lamp", price: 189, originalPrice: 240, category: "Li
  { id: 2, name: "Linen Accent Chair", price: 429, originalPrice: 550, category:
  { id: 3, name: "Ceramic Vase Set", price: 78, originalPrice: 95, category: "Dec
  { id: 4, name: "Walnut Floating Shelf", price: 145, originalPrice: 145, categor
  { id: 5, name: "Cashmere Throw Blanket", price: 215, originalPrice: 280, catego
  { id: 6, name: "Pour-Over Coffee Set", price: 94, originalPrice: 94, category:
  { id: 7, name: "Terrazzo Side Table", price: 320, originalPrice: 400, category:
  { id: 8, name: "Linen Duvet Cover", price: 162, originalPrice: 162, category: "
  { id: 9, name: "Brass Candle Holders", price: 55, originalPrice: 70, category:
  { id: 10, name: "Woven Storage Basket", price: 48, originalPrice: 48, category:
  { id: 11, name: "Oak Dining Table", price: 890, originalPrice: 1100, category:
  { id: 12, name: "Pendant Light Globe", price: 134, originalPrice: 160, category
  { id: 13, name: "Merino Wool Rug", price: 540, originalPrice: 680, category: "T
  { id: 14, name: "Sage Linen Cushion", price: 42, originalPrice: 42, category: "
  { id: 15, name: "Teak Soap Dish", price: 28, originalPrice: 35, category: "Kitc
];

const CATEGORIES = ["All", "Furniture", "Lighting", "Decor", "Textiles", "Kitchen

// ─── CONTEXT ─────────────────────────────────────────────────────────────────
const AppContext = createContext();

function AppProvider({ children }) {
  const [cart, setCart] = useState(() => {
    try { return JSON.parse(localStorage.getItem("cart") || "[]"); } catch { retu
  });
  const [wishlist, setWishlist] = useState([]);
  const [darkMode, setDarkMode] = useState(false);
  const [user, setUser] = useState(null);
  const [page, setPage] = useState("home");
  const [selectedProduct, setSelectedProduct] = useState(null);
  const [searchQuery, setSearchQuery] = useState("");
  const [selectedCategory, setSelectedCategory] = useState("All");
  const [sortBy, setSortBy] = useState("default");

  useEffect(() => {
    localStorage.setItem("cart", JSON.stringify(cart));
```

```
  }, [cart]);

  const addToCart = (product, qty = 1) => {
    setCart(prev => {
      const existing = prev.find(i => i.id === product.id);
      if (existing) return prev.map(i => i.id === product.id ? { ...i, qty: i.qty
      return [...prev, { ...product, qty }];
    });
  };

  const removeFromCart = (id) => setCart(prev => prev.filter(i => i.id !== id));

  const updateQty = (id, qty) => {
    if (qty < 1) return removeFromCart(id);
    setCart(prev => prev.map(i => i.id === id ? { ...i, qty } : i));
  };

  const toggleWishlist = (id) => {
    setWishlist(prev => prev.includes(id) ? prev.filter(w => w !== id) : [...prev
  };

  const cartCount = cart.reduce((s, i) => s + i.qty, 0);
  const cartTotal = cart.reduce((s, i) => s + i.price * i.qty, 0);

  return (
    <AppContext.Provider value={{
      cart, addToCart, removeFromCart, updateQty, cartCount, cartTotal,
      wishlist, toggleWishlist,
      darkMode, setDarkMode,
      user, setUser,
      page, setPage,
      selectedProduct, setSelectedProduct,
      searchQuery, setSearchQuery,
      selectedCategory, setSelectedCategory,
      sortBy, setSortBy,
    }}>
      {children}
    </AppContext.Provider>
  );
}

const useApp = () => useContext(AppContext);

// ─── COMPONENTS ──────────────────────────────────────────────────────────────

// Button Component
function Button({ children, variant = "primary", size = "md", onClick, className
```

```
  const base = "inline-flex items-center justify-center font-medium transition-al
  const variants = {
    primary: "bg-stone-900 text-white hover:bg-stone-700 focus:ring-stone-900",
    secondary: "bg-transparent border border-stone-900 text-stone-900 hover:bg-st
    ghost: "bg-transparent text-stone-600 hover:bg-stone-100 focus:ring-stone-400
    danger: "bg-red-600 text-white hover:bg-red-700 focus:ring-red-600",
  };
  const sizes = { sm: "px-3 py-1.5 text-sm rounded-md", md: "px-5 py-2.5 text-sm
  return (
    <button type={type} disabled={disabled} onClick={onClick}
      className={`${base} ${variants[variant]} ${sizes[size]} ${className}`}>
      {children}
    </button>
  );
}

// Input Field Component
function InputField({ label, type = "text", value, onChange, placeholder, icon, c
  return (
    <div className={`relative ${className}`}>
      {label && <label className="block text-xs font-semibold text-stone-500 uppe
      {icon && <span className="absolute left-3 top-1/2 -translate-y-1/2 text-sto
      <input
        type={type} value={value} onChange={onChange} placeholder={placeholder}
        className={`w-full border border-stone-200 rounded-xl bg-white px-4 py-3
      />
    </div>
  );
}

// Rating Component
function Rating({ value, count, size = "sm" }) {
  const starSize = size === "sm" ? "text-sm" : "text-base";
  return (
    <div className="flex items-center gap-1.5">
      <div className={`flex gap-0.5 ${starSize}`}>
        {[1,2,3,4,5].map(s => (
          <span key={s} className={s <= Math.round(value) ? "text-amber-400" : "t
        ))}
      </div>
      <span className="text-xs text-stone-500">{value} {count && `(${count})`}</s
    </div>
  );
}

// Badge Component
function Badge({ text }) {
```

```
  if (!text) return null;
  const colors = {
    "Best Seller": "bg-amber-100 text-amber-800",
    "New": "bg-emerald-100 text-emerald-800",
    "Sale": "bg-rose-100 text-rose-800",
  };
  return <span className={`text-xs font-semibold px-2 py-0.5 rounded-full ${color
}

// Loader Component
function Loader() {
  return (
    <div className="flex items-center justify-center py-24">
      <div className="w-8 h-8 border-2 border-stone-200 border-t-stone-800 rounde
    </div>
  );
}

// Breadcrumb Component
function Breadcrumb({ items }) {
  const { setPage } = useApp();
  return (
    <nav className="flex items-center gap-2 text-sm text-stone-500 mb-6">
      {items.map((item, i) => (
        <span key={i} className="flex items-center gap-2">
          {i > 0 && <span className="text-stone-300">/</span>}
          {item.page ? (
            <button onClick={() => setPage(item.page)} className="hover:text-ston
          ) : (
            <span className="text-stone-900 font-medium">{item.label}</span>
          )}
        </span>
      ))}
    </nav>
  );
}

// Search Bar Component
function SearchBar() {
  const { searchQuery, setSearchQuery, setPage, setSelectedCategory } = useApp();
  const [local, setLocal] = useState(searchQuery);

  const handleSubmit = (e) => {
    e.preventDefault();
    setSearchQuery(local);
    setSelectedCategory("All");
    setPage("products");
```

```
  };

  return (
    <form onSubmit={handleSubmit} className="relative">
      <svg className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-stone
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M21
      </svg>
      <input
        value={local} onChange={e => setLocal(e.target.value)}
        placeholder="Search products…"
        className="w-full pl-9 pr-4 py-2 text-sm border border-stone-200 rounded-
      />
    </form>
  );
}

// Category Filter Component
function CategoryFilter() {
  const { selectedCategory, setSelectedCategory, setSearchQuery } = useApp();
  return (
    <div className="flex flex-wrap gap-2">
      {CATEGORIES.map(cat => (
        <button key={cat} onClick={() => { setSelectedCategory(cat); setSearchQue
          className={`px-4 py-1.5 text-sm rounded-full border transition-all dura
            ? "bg-stone-900 text-white border-stone-900"
            : "bg-white text-stone-600 border-stone-200 hover:border-stone-400"
          }`}>{cat}</button>
      ))}
    </div>
  );
}

// Product Card Component
function ProductCard({ product }) {
  const { addToCart, wishlist, toggleWishlist, setPage, setSelectedProduct } = us
  const isWished = wishlist.includes(product.id);
  const discount = product.originalPrice > product.price
    ? Math.round((1 - product.price / product.originalPrice) * 100) : null;

  return (
    <div className="group bg-white rounded-2xl overflow-hidden border border-ston
      <div className="relative overflow-hidden aspect-square bg-stone-50">
        <img src={product.image} alt={product.name}
          className="w-full h-full object-cover group-hover:scale-105 transition-
        <div className="absolute top-3 left-3 flex flex-col gap-1">
          <Badge text={product.badge} />
          {discount && <span className="text-xs font-semibold bg-stone-900 text-w
```

```
        </div>
        <button onClick={() => toggleWishlist(product.id)}
          className="absolute top-3 right-3 w-8 h-8 rounded-full bg-white shadow
          {isWished ? "♥" : "♡"}
        </button>
        <div className="absolute inset-x-0 bottom-0 p-3 translate-y-full group-ho
          <Button onClick={() => addToCart(product)} className="w-full" size="sm"
        </div>
      </div>
      <div className="p-4">
        <p className="text-xs text-stone-400 uppercase tracking-widest mb-1">{pro
        <h3 className="font-semibold text-stone-800 mb-1 cursor-pointer hover:tex
          onClick={() => { setSelectedProduct(product); setPage("product"); }}>
          {product.name}
        </h3>
        <Rating value={product.rating} count={product.reviews} />
        <div className="flex items-center gap-2 mt-2">
          <span className="font-bold text-stone-900">${product.price}</span>
          {product.originalPrice > product.price && (
            <span className="text-xs text-stone-400 line-through">${product.origi
          )}
        </div>
      </div>
    </div>
  );
}

// Product Grid Component
function ProductGrid({ products }) {
  if (!products.length) return (
    <div className="col-span-full text-center py-24">
      <p className="text-4xl mb-3">🔍</p>
      <p className="text-stone-500 text-lg">No products found</p>
      <p className="text-stone-400 text-sm mt-1">Try adjusting your search or fil
    </div>
  );
  return (
    <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-4 md:gap-6
      {products.map(p => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}

// Cart Item Component
function CartItem({ item }) {
  const { updateQty, removeFromCart } = useApp();
  return (
```

```
    <div className="flex gap-4 py-5 border-b border-stone-100 last:border-0">
      <img src={item.image} alt={item.name} className="w-20 h-20 rounded-xl objec
      <div className="flex-1 min-w-0">
        <h4 className="font-semibold text-stone-800 truncate">{item.name}</h4>
        <p className="text-xs text-stone-400 uppercase tracking-widest">{item.cat
        <p className="text-sm font-bold text-stone-900 mt-1">${item.price}</p>
        <div className="flex items-center gap-3 mt-2">
          <div className="flex items-center border border-stone-200 rounded-lg ov
            <button onClick={() => updateQty(item.id, item.qty - 1)}
              className="w-8 h-8 flex items-center justify-center text-stone-500
            <span className="w-10 text-center text-sm font-medium">{item.qty}</sp
            <button onClick={() => updateQty(item.id, item.qty + 1)}
              className="w-8 h-8 flex items-center justify-center text-stone-500
          </div>
          <button onClick={() => removeFromCart(item.id)} className="text-xs text
        </div>
      </div>
      <p className="font-bold text-stone-900 flex-shrink-0">${(item.price * item.
    </div>
  );
}

// Cart Summary Component
function CartSummary() {
  const { cartTotal, setPage, cart } = useApp();
  const shipping = cartTotal > 300 ? 0 : 15;
  const tax = cartTotal * 0.08;

  return (
    <div className="bg-stone-50 rounded-2xl p-6 border border-stone-100">
      <h3 className="font-bold text-stone-800 text-lg mb-5">Order Summary</h3>
      <div className="space-y-3 text-sm">
        <div className="flex justify-between text-stone-600">
          <span>Subtotal ({cart.reduce((s,i)=>s+i.qty,0)} items)</span>
          <span>${cartTotal.toFixed(2)}</span>
        </div>
        <div className="flex justify-between text-stone-600">
          <span>Shipping</span>
          <span>{shipping === 0 ? <span className="text-emerald-600 font-medium">
        </div>
        <div className="flex justify-between text-stone-600">
          <span>Tax (8%)</span>
          <span>${tax.toFixed(2)}</span>
        </div>
        {cartTotal < 300 && (
          <p className="text-xs text-stone-400 bg-amber-50 border border-amber-10
            Add ${(300 - cartTotal).toFixed(0)} more for free shipping!
```

```
          </p>
        )}
        <div className="border-t border-stone-200 pt-3 flex justify-between font-
          <span>Total</span>
          <span>${(cartTotal + shipping + tax).toFixed(2)}</span>
        </div>
      </div>
      <Button onClick={() => alert("Checkout flow coming soon!")} className="w-fu
        Proceed to Checkout →
      </Button>
      <button onClick={() => setPage("products")}
        className="w-full mt-3 text-sm text-stone-500 hover:text-stone-800 transi
        ← Continue Shopping
      </button>
    </div>
  );
}

// Modal Component
function Modal({ isOpen, onClose, children }) {
  useEffect(() => {
    if (isOpen) document.body.style.overflow = "hidden";
    else document.body.style.overflow = "";
    return () => { document.body.style.overflow = ""; };
  }, [isOpen]);

  if (!isOpen) return null;
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
      <div className="absolute inset-0 bg-black/40 backdrop-blur-sm" onClick={onC
      <div className="relative bg-white rounded-3xl shadow-2xl max-w-md w-full p-
        <button onClick={onClose} className="absolute top-4 right-4 w-8 h-8 round
        {children}
      </div>
    </div>
  );
}

// Navbar Component
function Navbar() {
  const { cartCount, setPage, darkMode, setDarkMode, user, page } = useApp();
  const [mobileOpen, setMobileOpen] = useState(false);
  const navItems = [
    { label: "Home", page: "home" },
    { label: "Shop", page: "products" },
  ];
```

```
return (
  <header className="sticky top-0 z-40 bg-white/90 backdrop-blur-md border-b bo
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex items-center justify-between h-16">
        {/* Logo */}
        <button onClick={() => setPage("home")} className="font-bold text-xl tr
          <span className="text-stone-400 font-light">the</span>shelf
        </button>

        {/* Desktop Nav */}
        <nav className="hidden md:flex items-center gap-8">
          {navItems.map(item => (
            <button key={item.page} onClick={() => setPage(item.page)}
              className={`text-sm font-medium transition ${page === item.page ?
              {item.label}
            </button>
          ))}
        </nav>

        {/* Right actions */}
        <div className="flex items-center gap-2">
          <div className="hidden md:block w-48"><SearchBar /></div>
          <button onClick={() => setDarkMode(!darkMode)}
            className="w-9 h-9 rounded-full flex items-center justify-center te
            {darkMode ? "☀️" : "🌙"}
          </button>
          <button onClick={() => setPage("login")}
            className="w-9 h-9 rounded-full flex items-center justify-center te
            {user ? "👤" : "🔑"}
          </button>
          <button onClick={() => setPage("cart")}
            className="relative w-9 h-9 rounded-full flex items-center justify-
            🛒
            {cartCount > 0 && (
              <span className="absolute -top-0.5 -right-0.5 w-5 h-5 bg-stone-90
                {cartCount > 9 ? "9+" : cartCount}
              </span>
            )}
          </button>
          <button className="md:hidden w-9 h-9 flex items-center justify-center
            <div className="space-y-1.5">
              <span className="block w-5 h-0.5 bg-stone-700" />
              <span className="block w-5 h-0.5 bg-stone-700" />
              <span className="block w-3 h-0.5 bg-stone-700" />
            </div>
          </button>
        </div>
```

```
        </div>

        {/* Mobile Menu */}
        {mobileOpen && (
          <div className="md:hidden py-4 border-t border-stone-100 space-y-3">
            <SearchBar />
            {navItems.map(item => (
              <button key={item.page} onClick={() => { setPage(item.page); setMob
                className="block w-full text-left px-2 py-1.5 text-stone-700 font
                {item.label}
              </button>
            ))}
          </div>
        )}
      </div>
    </header>
  );
}

// Footer Component
function Footer() {
  const { setPage } = useApp();
  return (
    <footer className="bg-stone-900 text-stone-400 mt-24">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-16">
        <div className="grid grid-cols-2 md:grid-cols-4 gap-8 mb-12">
          <div className="col-span-2 md:col-span-1">
            <button onClick={() => setPage("home")} className="text-white font-bo
              <span className="text-stone-400 font-light">the</span>shelf
            </button>
            <p className="text-sm leading-relaxed">Curated objects for considered
          </div>
          {[
            { title: "Shop", links: ["All Products", "Furniture", "Lighting", "De
            { title: "Company", links: ["About Us", "Sustainability", "Careers",
            { title: "Support", links: ["FAQ", "Shipping", "Returns", "Contact"]
          ].map(col => (
            <div key={col.title}>
              <h4 className="text-white font-semibold text-sm mb-3">{col.title}</
              <ul className="space-y-2">
                {col.links.map(l => (
                  <li key={l}><button onClick={() => setPage("products")} classNa
                ))}
              </ul>
            </div>
          ))}
        </div>
```

```
        <div className="border-t border-stone-800 pt-8 flex flex-col md:flex-row
          <p className="text-xs">© 2025 TheShelf. All rights reserved.</p>
          <div className="flex gap-4 text-xs">
            <button className="hover:text-white transition">Privacy Policy</butto
            <button className="hover:text-white transition">Terms of Use</button>
          </div>
        </div>
      </div>
    </footer>
  );
}

// Hero Section Component
function HeroSection() {
  const { setPage } = useApp();
  return (
    <section className="relative overflow-hidden bg-stone-50 min-h-[85vh] flex it
      {/* Background grid pattern */}
      <div className="absolute inset-0 opacity-30"
        style={{ backgroundImage: "radial-gradient(circle, #d6d3d1 1px, transpare

      <div className="relative max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-24 w-ful
        <div className="grid lg:grid-cols-2 gap-16 items-center">
          <div>
            <span className="inline-block text-xs font-bold uppercase tracking-[0
              Spring Collection 2025
            </span>
            <h1 className="text-5xl sm:text-6xl lg:text-7xl font-bold text-stone-
              Objects<br />worth<br /><span className="italic font-light text-sto
            </h1>
            <p className="text-lg text-stone-500 max-w-md mb-8 leading-relaxed">
              A thoughtfully curated collection of furniture, lighting, and décor
            </p>
            <div className="flex flex-col sm:flex-row gap-3">
              <Button onClick={() => setPage("products")} size="lg">Shop Now →</B
              <Button onClick={() => setPage("products")} variant="secondary" siz
            </div>
            <div className="flex gap-8 mt-12 pt-8 border-t border-stone-200">
              {[["2,400+", "Products"], ["48h", "Dispatch"], ["Free", "Returns"]]
                <div key={label}>
                  <p className="text-2xl font-bold text-stone-900">{val}</p>
                  <p className="text-xs text-stone-400 uppercase tracking-widest"
                </div>
              ))}
            </div>
          </div>
          <div className="hidden lg:grid grid-cols-2 gap-4">
```

```
            {PRODUCTS.slice(0, 4).map((p, i) => (
              <div key={p.id}
                style={{ marginTop: i % 2 === 1 ? "2rem" : "0" }}
                className="rounded-2xl overflow-hidden aspect-square bg-stone-100
                <img src={p.image} alt={p.name} className="w-full h-full object-c
              </div>
            ))}
          </div>
        </div>
      </div>
    </section>
  );
}

// ─── PAGES ──────────────────────────────────────────────────────────────────

// Home Page
function HomePage() {
  const { setPage, setSelectedCategory } = useApp();
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const t = setTimeout(() => setLoading(false), 600);
    return () => clearTimeout(t);
  }, []);

  if (loading) return <Loader />;

  return (
    <div>
      <HeroSection />

      {/* Categories */}
      <section className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-20">
        <div className="text-center mb-12">
          <h2 className="text-3xl font-bold text-stone-900 mb-2">Shop by Category
          <p className="text-stone-500">Explore our curated collections</p>
        </div>
        <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-5 gap-4">
          {CATEGORIES.filter(c => c !== "All").map((cat, i) => {
            const imgs = { Furniture: PRODUCTS[1].image, Lighting: PRODUCTS[0].im
            return (
              <button key={cat}
                onClick={() => { setSelectedCategory(cat); setPage("products"); }
                className="group relative overflow-hidden rounded-2xl aspect-squa
                <img src={imgs[cat]} alt={cat} className="w-full h-full object-co
                <div className="absolute inset-0 bg-gradient-to-t from-black/60 v
```

```
                <p className="absolute bottom-4 left-0 right-0 text-center text-w
              </button>
            );
          })}
        </div>
      </section>

      {/* Featured Products */}
      <section className="bg-stone-50 py-20">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex items-center justify-between mb-10">
            <div>
              <h2 className="text-3xl font-bold text-stone-900">Best Sellers</h2>
              <p className="text-stone-500 mt-1">Our most loved pieces right now<
            </div>
            <Button onClick={() => setPage("products")} variant="secondary">View
          </div>
          <ProductGrid products={PRODUCTS.filter(p => p.badge === "Best Seller" |
        </div>
      </section>

      {/* Banner */}
      <section className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-20">
        <div className="bg-stone-900 rounded-3xl p-12 md:p-16 text-center text-wh
          <div className="absolute inset-0 opacity-10"
            style={{ backgroundImage: "radial-gradient(circle, #fff 1px, transpar
          <p className="text-stone-400 text-sm uppercase tracking-widest mb-3">Li
          <h2 className="text-4xl md:text-5xl font-bold mb-4">Up to 30% off</h2>
          <p className="text-stone-400 mb-8 text-lg">On selected furniture and li
          <Button onClick={() => setPage("products")} variant="secondary" size="l
            className="!border-white !text-white !hover:bg-white/10">
            Shop the Sale →
          </Button>
        </div>
      </section>
    </div>
  );
}

// Product Listing Page
function ProductsPage() {
  const { searchQuery, setSearchQuery, selectedCategory, sortBy, setSortBy } = us
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);
    const t = setTimeout(() => setLoading(false), 400);
```

```
  return () => clearTimeout(t);
}, [selectedCategory, searchQuery]);

let filtered = PRODUCTS;
if (selectedCategory !== "All") filtered = filtered.filter(p => p.category ===
if (searchQuery) filtered = filtered.filter(p =>
  p.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
  p.category.toLowerCase().includes(searchQuery.toLowerCase())
);
if (sortBy === "price-asc") filtered = [...filtered].sort((a, b) => a.price - b
if (sortBy === "price-desc") filtered = [...filtered].sort((a, b) => b.price -
if (sortBy === "rating") filtered = [...filtered].sort((a, b) => b.rating - a.r
if (sortBy === "newest") filtered = [...filtered].filter(p => p.badge === "New"

return (
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-12">
    <Breadcrumb items={[{ label: "Home", page: "home" }, { label: "All Products

    <div className="flex flex-col md:flex-row md:items-center justify-between g
      <div>
        <h1 className="text-3xl font-bold text-stone-900">
          {searchQuery ? `Results for "${searchQuery}"` : selectedCategory ===
        </h1>
        <p className="text-stone-500 text-sm mt-1">{filtered.length} product{fi
      </div>
      <div className="flex items-center gap-3">
        {searchQuery && (
          <button onClick={() => setSearchQuery("")}
            className="text-xs text-stone-500 hover:text-stone-900 bg-stone-100
            Clear search ×
          </button>
        )}
        <select value={sortBy} onChange={e => setSortBy(e.target.value)}
          className="text-sm border border-stone-200 rounded-xl px-4 py-2 bg-wh
          <option value="default">Sort: Featured</option>
          <option value="price-asc">Price: Low to High</option>
          <option value="price-desc">Price: High to Low</option>
          <option value="rating">Best Rated</option>
          <option value="newest">New Arrivals</option>
        </select>
      </div>
    </div>

    <div className="mb-8 overflow-x-auto">
      <CategoryFilter />
    </div>
```

```
      {loading ? <Loader /> : <ProductGrid products={filtered} />}
    </div>
  );
}

// Product Detail Page
function ProductDetailPage() {
  const { selectedProduct: product, addToCart, wishlist, toggleWishlist, setPage,
  const [qty, setQty] = useState(1);
  const [tab, setTab] = useState("description");
  const [added, setAdded] = useState(false);

  if (!product) { setPage("products"); return null; }

  const isWished = wishlist.includes(product.id);
  const related = PRODUCTS.filter(p => p.category === product.category && p.id !=

  const handleAddToCart = () => {
    addToCart(product, qty);
    setAdded(true);
    setTimeout(() => setAdded(false), 2000);
  };

  return (
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-12">
      <Breadcrumb items={[
        { label: "Home", page: "home" },
        { label: "Shop", page: "products" },
        { label: product.category, page: "products" },
        { label: product.name },
      ]} />

      <div className="grid lg:grid-cols-2 gap-12 mb-20">
        {/* Image */}
        <div className="relative rounded-3xl overflow-hidden aspect-square bg-sto
          <img src={product.image} alt={product.name} className="w-full h-full ob
          <div className="absolute top-4 left-4 flex flex-col gap-2">
            <Badge text={product.badge} />
          </div>
          <button onClick={() => toggleWishlist(product.id)}
            className="absolute top-4 right-4 w-10 h-10 rounded-full bg-white sha
            {isWished ? "♥" : "♡"}
          </button>
        </div>

        {/* Info */}
        <div className="flex flex-col">
```

```
<p className="text-xs font-semibold uppercase tracking-[0.2em] text-sto
<h1 className="text-3xl md:text-4xl font-bold text-stone-900 mb-3">{pro
<Rating value={product.rating} count={product.reviews} size="md" />

<div className="flex items-center gap-3 my-5">
  <span className="text-3xl font-bold text-stone-900">${product.price}<
  {product.originalPrice > product.price && (
    <>
      <span className="text-lg text-stone-400 line-through">${product.o
      <span className="text-sm font-semibold text-rose-600 bg-rose-50 p
        Save ${product.originalPrice - product.price}
      </span>
    </>
  )}
</div>

{/* Tabs */}
<div className="flex gap-4 border-b border-stone-200 mb-5">
  {["description", "details", "shipping"].map(t => (
    <button key={t} onClick={() => setTab(t)}
      className={`pb-2 text-sm font-medium capitalize transition border
      {t}
    </button>
  ))}
</div>
<div className="text-stone-600 text-sm leading-relaxed mb-6 min-h-[60px
  {tab === "description" && product.description}
  {tab === "details" && "Material: Premium quality. Origin: Artisan-mad
  {tab === "shipping" && "Free shipping on orders over $300. Standard d
</div>

{/* Quantity */}
<div className="flex items-center gap-4 mb-5">
  <span className="text-sm font-semibold text-stone-700">Quantity</span
  <div className="flex items-center border border-stone-200 rounded-xl
    <button onClick={() => setQty(Math.max(1, qty - 1))}
      className="w-10 h-10 flex items-center justify-center text-stone-
    <span className="w-12 text-center font-medium">{qty}</span>
    <button onClick={() => setQty(qty + 1)}
      className="w-10 h-10 flex items-center justify-center text-stone-
  </div>
</div>

<div className="flex gap-3">
  <Button onClick={handleAddToCart} size="lg" className="flex-1">
    {added ? "✓ Added!" : "Add to Cart"}
  </Button>
```

```
            <Button onClick={() => toggleWishlist(product.id)} variant="secondary
              {isWished ? "♥" : "♡"}
            </Button>
          </div>

          <div className="mt-6 pt-6 border-t border-stone-100 grid grid-cols-3 ga
            {[["🚚", "Free Shipping", "Orders $300+"], ["↩️", "Free Returns", "30
              <div key={title}>
                <p className="text-xl mb-1">{icon}</p>
                <p className="text-xs font-semibold text-stone-700">{title}</p>
                <p className="text-xs text-stone-400">{sub}</p>
              </div>
            ))}
          </div>
        </div>
      </div>

      {/* Related Products */}
      {related.length > 0 && (
        <section>
          <h2 className="text-2xl font-bold text-stone-900 mb-8">You Might Also L
          <ProductGrid products={related} />
        </section>
      )}
    </div>
  );
}

// Cart Page
function CartPage() {
  const { cart, setPage, cartCount } = useApp();

  if (cartCount === 0) return (
    <div className="max-w-2xl mx-auto px-4 py-32 text-center">
      <p className="text-6xl mb-6">🛒</p>
      <h2 className="text-3xl font-bold text-stone-900 mb-3">Your cart is empty</
      <p className="text-stone-500 mb-8">Looks like you haven't added anything ye
      <Button onClick={() => setPage("products")} size="lg">Start Shopping</Butto
    </div>
  );

  return (
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-12">
      <Breadcrumb items={[{ label: "Home", page: "home" }, { label: "Cart" }]} />
      <h1 className="text-3xl font-bold text-stone-900 mb-10">Your Cart ({cartCou

      <div className="grid lg:grid-cols-3 gap-10">
```

```
        <div className="lg:col-span-2 bg-white rounded-2xl border border-stone-10
          {cart.map(item => <CartItem key={item.id} item={item} />)}
        </div>
        <div>
          <CartSummary />
        </div>
      </div>
    </div>
  );
}

// Login / Signup Page
function LoginPage() {
  const { setUser, setPage } = useApp();
  const [mode, setMode] = useState("login");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [name, setName] = useState("");
  const [success, setSuccess] = useState(false);

  const handleSubmit = (e) => {
    e.preventDefault();
    setUser({ name: name || email.split("@")[0], email });
    setSuccess(true);
    setTimeout(() => setPage("home"), 1500);
  };

  if (success) return (
    <div className="min-h-[70vh] flex items-center justify-center">
      <div className="text-center">
        <p className="text-5xl mb-4">✓</p>
        <h2 className="text-2xl font-bold text-stone-900 mb-2">{mode === "login"
        <p className="text-stone-500">Redirecting you home…</p>
      </div>
    </div>
  );

  return (
    <div className="min-h-[80vh] flex items-center justify-center px-4 py-20">
      <div className="w-full max-w-md">
        <div className="text-center mb-10">
          <h1 className="text-4xl font-bold text-stone-900 mb-2">
            {mode === "login" ? "Welcome back" : "Create account"}
          </h1>
          <p className="text-stone-500">
            {mode === "login" ? "Sign in to your account" : "Start your journey w
          </p>
```

```
        </div>

        {/* Toggle */}
        <div className="flex bg-stone-100 rounded-2xl p-1 mb-8">
          {["login", "signup"].map(m => (
            <button key={m} onClick={() => setMode(m)}
              className={`flex-1 py-2.5 text-sm font-medium rounded-xl transition
              {m === "login" ? "Sign In" : "Create Account"}
            </button>
          ))}
        </div>

        <form onSubmit={handleSubmit} className="space-y-4">
          {mode === "signup" && (
            <InputField label="Full Name" value={name} onChange={e => setName(e.t
          )}
          <InputField label="Email Address" type="email" value={email} onChange={
          <InputField label="Password" type="password" value={password} onChange=
          {mode === "login" && (
            <div className="text-right">
              <button type="button" className="text-xs text-stone-500 hover:text-
            </div>
          )}
          <Button type="submit" size="lg" className="w-full !mt-6">
            {mode === "login" ? "Sign In →" : "Create Account →"}
          </Button>
        </form>

        <div className="relative my-8">
          <div className="border-t border-stone-200" />
          <span className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-
        </div>

        <div className="grid grid-cols-2 gap-3">
          {["Google", "Apple"].map(provider => (
            <button key={provider} type="button"
              className="flex items-center justify-center gap-2 border border-sto
              {provider === "Google" ? "G" : "🍎"} {provider}
            </button>
          ))}
        </div>
      </div>
    </div>
  );
}

// ─── APP SHELL ──────────────────────────────────────────────────────────
```

```
function AppShell() {
  const { page, darkMode } = useApp();

  const pageMap = {
    home: <HomePage />,
    products: <ProductsPage />,
    product: <ProductDetailPage />,
    cart: <CartPage />,
    login: <LoginPage />,
  };

  return (
    <div className={darkMode ? "dark" : ""}>
      <div className="min-h-screen bg-white text-stone-900 font-sans">
        <Navbar />
        <main>
          {pageMap[page] || <HomePage />}
        </main>
        <Footer />
      </div>
    </div>
  );
}

export default function App() {
  return (
    <AppProvider>
      <AppShell />
    </AppProvider>
  );
}
```